Task 1:

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```
  GNU nano 7.2                                producer_consumer.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

int main() {
    int pipefd[2];
    int data;
    pid_t pid;

    pipe(pipefd);

    pid = fork();

    if (pid > 0) {
        // Parent = Producer
        close(pipefd[0]);

        clock_t start = clock();

        for (int i = 1; i <= 5; i++) {
            write(pipefd[1], &i, sizeof(i));
            printf("Produced: %d\n", i);
        }

        close(pipefd[1]);
        wait(NULL);

        clock_t end = clock();

        printf("Time taken: %f seconds\n",
               (double)(end - start) / CLOCKS_PER_SEC);
    } else {
        // Child = Consumer
        close(pipefd[1]);

        while (read(pipefd[0], &data, sizeof(data)) > 0) {
            printf("Consumed: %d\n", data);
        }

        close(pipefd[0]);
    }
    return 0;
}
```

Output:

```
suraj-ubuntu@suraj-ubuntu:~$ nano producer_consumer.c
suraj-ubuntu@suraj-ubuntu:~$ gcc producer_consumer.c -o producer_consumer
suraj-ubuntu@suraj-ubuntu:~$ ./producer_consumer
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Produced: 5
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
Time taken: 0.698178 seconds
suraj-ubuntu@suraj-ubuntu:~$
```

Observation:

The parent process acts as the producer and generates integer values from 1 to 5. These values are sent to the child process through an anonymous pipe using the write() system call. The child process acts as the consumer and receives the values using the read() system call. The communication time is measured to determine the efficiency of data transfer between the two processes.

Result:

The producer-consumer communication system was successfully implemented using an anonymous pipe, and the communication time was measured successfully.

Task 2:

Develop a program that executes the equivalent of the shell command ls -l | grep ".c" using fork(), pipe(), dup2(), and exec() system calls.

```
 GNU nano 7.2                                pipe_ls.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];

    pipe(pipefd);

    if (fork() == 0) {
        // Child 1: ls -l
        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);
    }
    else {
        // Parent creates second child
        if (fork() == 0) {
            // Child 2: grep ".c"
            close(pipefd[1]);

            dup2(pipefd[0], STDIN_FILENO);
            close(pipefd[0]);

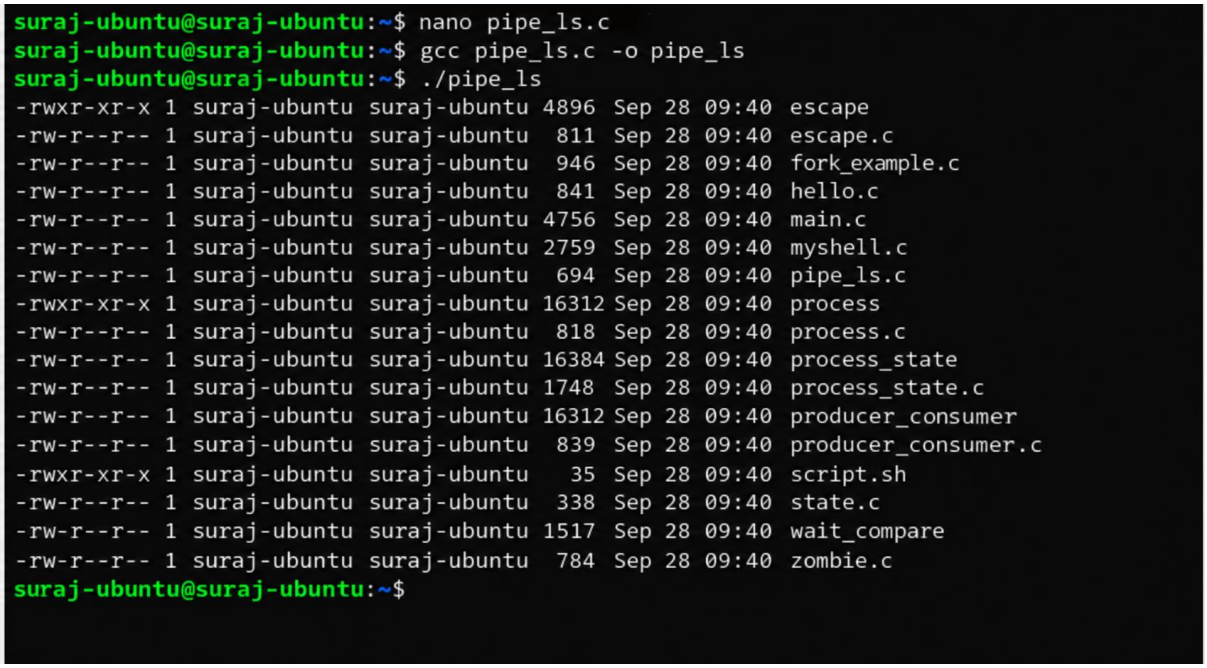
            execlp("grep", "grep", ".c", NULL);
        }

        close(pipefd[0]);
        close(pipefd[1]);

        wait(NULL);
        wait(NULL);
    }

    return 0;
}
```

Output:



Observation:

The program successfully implements the shell pipeline using fork(), pipe(), dup2(), and exec() system calls. The output of ls -l is redirected through the pipe to grep ".c", which displays the matching entries.

Result:

The shell command ls -l | grep ".c" was successfully implemented using system calls.